



UNICEPLAC
CENTRO UNIVERSITÁRIO

LabHub UNICEPLAC

Central de Reservas e Gestão de Laboratórios Acadêmicos

Documentação Técnica Completa

Versão 1.0 • 18 de junho de 2026

Sumário

1. Introdução e Objetivo
2. Stack Tecnológico e Dependências
3. Arquitetura
4. Estrutura de Diretórios
5. Configuração
6. Roteamento
7. Autenticação e Autorização
8. Segurança
9. Modelo de Dados
10. Módulos Funcionais por Perfil
11. Relatórios BI (Business Intelligence)
12. Controle de Chaves
13. Chamados e Alerta Sonoro
14. Helpers
15. Camada de Dados (DAO / Model)
16. Front-end
17. Scripts de Banco de Dados
18. Instalação e Execução
19. Operação e Troubleshooting
20. Glossário

1. Introdução e Objetivo

O **LabHub UNICEPLAC** é um sistema web para gestão centralizada dos laboratórios acadêmicos da instituição. Ele organiza desde a solicitação e aprovação de reservas até o quadro de horários semanal, o ensalamento de turmas, o controle de entrega de chaves, a abertura de chamados técnicos e a geração de relatórios analíticos (BI).

O sistema atende três perfis de usuário, cada um com seu próprio painel:

- **Professor** — solicita reservas, acompanha suas aulas e aciona o suporte.
- **Coordenação** — aprova reservas, monta o quadro de horários e analisa indicadores.
- **Suporte Técnico** — controla chaves, atende chamados e acompanha a ocupação diária.

Objetivos principais:

- Reduzir conflitos de agendamento com verificação automática de choque de horário.
- Centralizar a operação dos laboratórios numa única ferramenta.
- Melhorar a comunicação entre professores, coordenação e suporte.
- Fornecer indicadores de ocupação e uso para apoiar a tomada de decisão.

2. Stack Tecnológico e Dependências

O back-end é escrito em PHP puro seguindo o padrão MVC; o front-end usa HTML, CSS e JavaScript com bibliotecas via CDN. A persistência é em MySQL via PDO.

Camada	Tecnologia
Linguagem	PHP 8.2+ (validado em PHP 8.5)
Banco de dados	MySQL 8.x — PDO, charset utf8mb4, engine InnoDB
Padrão arquitetural	MVC + Front Controller, com camadas DAO e Service
Front-end	HTML5, CSS3, JavaScript (vanilla)
UI / Componentes	Bootstrap 5.3, Bootstrap Icons 1.11
Gráficos (BI)	Chart.js 4.4
Calendário	FullCalendar 6.1
Autenticação	Sessão PHP + Google OAuth 2.0
Pacotes	Composer

Dependências declaradas no `composer.json`:

Pacote	Versão	Uso no projeto
google/apiclient	^2.15	Login social via Google OAuth 2.0
vlucas/phpdotenv	^5.6	Leitura de variáveis de ambiente do <code>.env</code>
phpmailer/phpmailer	^6.9	Biblioteca de e-mail (disponível p/ verificação)
monolog/monolog	^2.9	Biblioteca de logs (disponível)

Nota: PHPMailer e Monolog estão declarados como dependências, mas o envio de e-mail e o logging estruturado não estão ativados no fluxo atual — ficam disponíveis para evolução futura.

3. Arquitetura

A aplicação tem um **único ponto de entrada** público (`public/index.php`). Toda requisição passa por ele, que inicia a sessão, valida o token CSRF (em POST) e despacha para o controller correspondente segundo a tabela de rotas.

Fluxo de uma requisição:

```

Requisição HTTP (?page=...)
    |
    v
public/index.php --> backend/routes/web.php (page => [Controller, metodo])
    |
    v
Controller --> Service / Model --> DAO (interface) --> DAOImpl (PDO) --> MySQL
    |
    v
View (frontend/views/*.php) --> HTML + Bootstrap + Chart.js / FullCalendar
  
```

Responsabilidades das camadas:

Camada	Responsabilidade
Front Controller	Sessão, CSRF, roteamento e despacho (<code>public/index.php</code>)
Rotas	Mapa <code>?page=</code> → <code>[Controller, método]</code> (<code>backend/routes/web.php</code>)
Controllers	Orquestram a requisição e renderizam a view; estendem <code>BaseController</code>
Services	Regra de negócio reutilizável (ex.: <code>CadastroRestService</code>)
Models	Operações de domínio (<code>Agendamento</code> , <code>ControleChave</code> , <code>ChamadoSuporte...</code>)
DAO / DAOImpl	Contratos e implementações PDO de acesso a dados
Helpers	<code>Auth</code> , <code>Csrf</code> , <code>Upload</code> , <code>HorarioHelper</code> , <code>EnsalamentoHelper</code>
Views	Templates PHP por perfil + layout unificado

Padrões de projeto aplicados:

- **Front Controller** — um único arquivo trata todas as requisições.
- **MVC** — separação entre dados (Model/DAO), apresentação (View) e controle (Controller).
- **DAO + Interface** — desacopla o acesso a dados das implementações concretas.
- **PRG (Post/Redirect/Get)** — evita reenvio de formulários ao recarregar a página.
- **Injeção de dependência simples** — o PDO é injetado nos controllers e DAOs.

4. Estrutura de Diretórios

```
labhubuniceplac/
|-- public/                # Document root (unico ponto publico)
|   |-- index.php          # Front Controller
|   |-- .htaccess          # Rewrite -> index.php
|   |-- assets/            # css/ js/ images/ sounds/
|   |-- uploads/           # Fotos de perfil enviadas
|
|-- backend/
|   |-- config/             # app.php (constantes/.env) e database.php (PDO)
|   |-- routes/web.php      # Tabela de rotas
|   |-- controllers/        # Auth, Professor, Coordenador, Suporte, Api, Cadastro
|   |-- services/           # CadastroRestService
|   |-- models/             # Agendamento, ControleChave, ChamadoSuporte, ...
|   |-- DAOs/               # Interfaces (contratos)
|   |-- DAOImpl/            # Implementacoes PDO
|   |-- helpers/            # Auth, Csrft, Upload, HorarioHelper, EnsalamentoHelper
|   |-- database/           # setup.php, seed.php, seed_bi.php, migrate.php
|
|-- frontend/views/         # auth/ professor/ coordenador/ suporte/ shared/ layouts/
|-- docker/entrypoint.sh    # Bootstrap do container
|-- composer.json / .lock
|-- .env.example
`-- README.md
```

5. Configuração

backend/config/app.php

Carrega o autoloader do Composer, lê o `.env` (via `phpdotenv`), define o fuso `America/Sao_Paulo`, ajusta o nível de erros e declara as constantes globais:

Constante	Valor / Função
<code>ROOT_PATH / BACKEND_PATH / VIEWS_PATH</code>	Caminhos-base da aplicação
<code>APP_NAME</code>	"UNICEPLAC - Central de Reservas"
<code>UPLOAD_DIR / UPLOAD_URL</code>	Diretório e URL das fotos de perfil
<code>DEFAULT_AVATAR</code>	Imagem padrão de avatar
<code>APP_DEBUG</code> (env)	Controla <code>display_errors</code> (tela vs. log)

backend/config/database.php

Cria a conexão PDO com MySQL usando as variáveis do ambiente, define o fuso do banco para `-03:00`, ativa exceções (`ERRMODE_EXCEPTION`) e o fetch associativo por padrão. Em caso de falha, responde HTTP 500 com JSON de erro.

```
$pdo = new PDO("mysql:host=$host;port=$port;dbname=$dbname;charset=utf8mb4", $user, $pass);  
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
```

6. Roteamento

O front controller lê `?page=`, consulta a tabela em `backend/routes/web.php` e instancia o controller. Rotas desconhecidas caem em `login`. O `.htaccess` reescreve URLs amigáveis para o front controller.

Rota (?page=)	Controller → método	Descrição
login	AuthController::login	Login por e-mail/senha
cadastro	AuthController::cadastro	Cadastro de professor
verificar	AuthController::verificar	Verificação de e-mail por token
google	AuthController::google	Login via Google OAuth
logout	AuthController::logout	Encerra a sessão
professor	ProfessorController::index	Painel do professor
coordenador	CoordenadorController::index	Painel da coordenação
kanban	CoordenadorController::kanban	Mover aula (AJAX drag-drop)
suporte	SuporteController::index	Painel do suporte
api/check-sos	ApiController::checkSos	Contagem de chamados (JSON)
api/check-sos-status	ApiController::checkSosStatus	Chamados pendentes + HTML (JSON)
api/andares	ApiController::andaresPorBloco	Andares de um bloco (JSON)
api/salas	ApiController::salasPorAndar	Salas de um andar (JSON)

7. Autenticação e Autorização

Login por e-mail e senha

O `AuthController::login` busca o usuário por e-mail (`UsuarioDAOImpl::findByEmail`) e valida a senha com `password_verify`. Contas com `email_verificado = 0` são bloqueadas. No sucesso, `Auth::login` regenera o ID de sessão e o usuário é redirecionado ao painel do seu perfil.

Cadastro

- Exige e-mail institucional terminando em `@uniceplac.edu.br`.
- Senha mínima de 8 caracteres e confirmação de senha.
- A senha é gravada com hash (`password_hash`).
- Novos usuários recebem o perfil **professor** e já entram com e-mail verificado.

Login com Google (OAuth 2.0)

O `AuthController::google` usa a biblioteca `google/apiclient`. A `redirect URI` é derivada do host atual (funciona em localhost ou atrás de túnel/proxy) ou fixada via `GOOGLE_REDIRECT_URI`. Opcionalmente restringe o acesso a um domínio (`GOOGLE_ALLOWED_DOMAIN`). Na volta, `upsertGoogle` cria ou atualiza o usuário.

Verificação de e-mail

A rota `verificar` valida um `token_verificacao` e ativa a conta. O fluxo está presente como estrutura; no cadastro atual a conta já é criada ativa.

Sessão e controle de acesso

O `helper Auth` centraliza sessão (`id`, `nome`, `perfil`, `foto`). O `BaseController::requireAuth($perfil)` protege cada painel: sem sessão → login; perfil errado → redireciona ao painel correto via `Auth::destinoAposLogin`.

Perfil	Painel padrão após login
coordenador	<code>/index.php?page=coordenador</code>
suporte	<code>/index.php?page=suporte</code>
professor (default)	<code>/index.php?page=professor</code>

8. Segurança

Proteção CSRF

O helper `Csrf` gera um token por sessão (32 bytes aleatórios). O front controller chama `Csrf::check()` em **toda** requisição POST, comparando o token da sessão com o enviado no corpo (`_csrf`) ou no header `X-CSRF-Token` com `hash_equals`. Token inválido encerra com HTTP 419.

O token é exposto numa `<meta name="csrf-token">` e injetado automaticamente em todo formulário POST no momento do submit (via `assets/js/app.js`), cobrindo inclusive formulários inseridos dinamicamente (ex.: chamados via polling).

Demais controles

- **Senhas** com hash `bcrypt` (`password_hash` / `password_verify`).
- **Cookie de sessão** `HttpOnly`, `SameSite=Lax` e `Secure` sob HTTPS.
- **`session_regenerate_id`** no login (mitiga fixation).
- **`requireAuth`** com checagem de perfil em todas as rotas de painel.
- **Upload** restrito a `jpg/jpeg/png/webp`, com nome derivado do usuário+timestamp.
- **Validação de posse**: a retirada de chave confere se a reserva é do professor logado.
- **`.htaccess`** desativa listagem de diretórios (`Options -Indexes`).
- **`APP_DEBUG=false`** oculta erros da tela em produção (mantém no log).

9. Modelo de Dados

Banco `sistema_labs` (MySQL/InnoDB, `utf8mb4`). O `schema` é criado por `backend/database/setup.php`.

Tabelas e finalidade:

Tabela	Finalidade	Status / Enums
<code>usuarios</code>	Contas e perfis, foto, verificação	<code>perfil</code> ; <code>email_verificado</code>
<code>laboratorios</code>	Laboratórios e capacidade	—
<code>disciplinas</code>	Disciplinas (nome único)	—
<code>cursos</code>	Cursos (nome único)	—
<code>semestres</code>	Semestres/turmas (nome único)	—
<code>blocos / andares / salas</code>	Hierarquia física dos espaços	—
<code>agendamentos</code>	Reservas avulsas	<code>pendente/aprovado/rejeitado/cancelado</code>
<code>chamados_suporte</code>	Chamados técnicos	<code>pendente/resolvido</code>
<code>controle_chaves</code>	Retirada/devolução de chaves	<code>em_uso/devolvido</code>
<code>quadros_horarios</code>	Versões do quadro (período letivo)	—
<code>quadro_aulas</code>	Aulas fixas da grade (alimenta o BI)	<code>turno</code> ; <code>dia_semana</code>
<code>ensalamento</code>	Alocação de turmas em salas	<code>turno</code>

Relacionamentos principais (chaves estrangeiras):

- `blocos` 1–N `andares` 1–N `salas` (FK `RESTRICT`; nome único por pai).
- `agendamentos` → `laboratorios`, `usuarios`, `disciplinas` (`RESTRICT`); chave única (`lab`, `data`, `turno`, `período`) evita reserva duplicada.
- `quadro_aulas` → `quadros_horarios` (`CASCADE`), `disciplinas/cursos/semestres` (`RESTRICT`), `professor/laboratório/sala` (`SET NULL`).
- `ensalamento` → `usuarios` (`CASCADE`), `sala` (`RESTRICT`); única por (`sala`, `turno`).
- `chamados_suporte` → `usuarios` (`CASCADE`).
- `controle_chaves` — **sem FK** em `id_agendamento`, pois também guarda ids sintéticos (+1.000.000) das aulas da grade fixa.

Detalhe: tabela `quadro_aulas`

É o coração do quadro de horários e a fonte dos Relatórios BI. Campos relevantes:

Campo	Tipo	Observação
<code>id_quadro</code>	INT	FK → <code>quadros_horarios</code> (<code>CASCADE</code>)
<code>id_professor</code>	INT NULL	FK → <code>usuarios</code> (<code>SET NULL</code>); NULL = EAD
<code>id_laboratorio</code>	INT NULL	FK → <code>laboratorios</code> (<code>SET NULL</code>)
<code>id_curso / id_semestre / id_disciplina</code>	INT	FKs obrigatórias
<code>id_sala</code>	INT NULL	FK → <code>salas</code> (<code>SET NULL</code>)

Campo	Tipo	Observação
turno	ENUM	Matutino/Vespertino/Noturno
dia_semana	VARCHAR	Segunda ... Sábado
carga_horaria_total	INT	Horas semanais (base do BI por professor/curso)
horas_laboratorio	INT	Horas em laboratório (base de ocupação)

10. Módulos Funcionais por Perfil

Professor — `ProfessorController::index`

Concentra as ações do professor; cada ação é um POST tratado e seguido de redirect (PRG). Principais ações:

- **Solicitar reserva** — valida choque de horário (`HorarioHelper::verificaChoque`) antes de gravar; reserva duplicada (lab/dia/turno) é barrada pela chave única.
- **Retirada de chave** — confere posse da reserva e janela de horário do turno (Matutino 07:00–12:30, Vespertino 13:00–18:30, Noturno 18:00–23:00).
- **SOS** — abre chamado durante aula em andamento.
- **Chamado geral** — abre chamado informando local e descrição.
- **Foto de perfil** — upload validado.

Coordenação — `CoordenadorController::index`

- Aprovar/rejeitar solicitações e gerenciar reservas.
- Quadro de horários (CRUD de aulas) e Kanban (mover aula entre dias via AJAX → rota kanban).
- Ensalamento (alocação por sala/turno) com selects encadeados bloco→andar→sala (rotas api/andares e api/salas).
- Cadastros de cursos, semestres, blocos, andares, salas, laboratórios e disciplinas.
- Calendário consolidado (reservas + grade fixa + feriados).
- Relatórios BI (seção 11).

Suporte — `SuporteController::index`

- Mapa diário de ocupação por data (reservas aprovadas + grade fixa do dia).
- Controle de chaves: dar baixa (devolução), destaque de atrasos e histórico.
- Chamados: fila de pendentes atualizada por polling, com alerta sonoro (seção 13).
- Mapa de ensalamento e relação de laboratórios.

11. Relatórios BI (Business Intelligence)

Calculados em `CoordenadorController::calcularRelatoriosBI()` a partir das aulas do **quadro selecionado** (`quadro_aulas`). Se não houver quadro, os indicadores ficam zerados.

Indicadores (KPIs)

KPI	Como é calculado
Taxa de Ocupação	$\text{uso total} \div \text{capacidade global} \times 100$
Horas Totais de Uso	soma de <code>horas_laboratorio</code> de todos os labs
Lab Mais Usado	laboratório com maior soma de <code>horas_laboratorio</code>
Lab Mais Ocioso	laboratório com maior ociosidade ($\text{capacidade} - \text{uso}$)

Capacidade do laboratório

A ocupação usa uma **janela operacional semanal** fixa de 60 horas por laboratório ($\approx 6 \text{ dias} \times 3 \text{ turnos}$). É uma medida de **tempo** — por isso não se usa a capacidade de assentos, que mede pessoas, não horas.

Gráficos (Chart.js)

Gráfico	Tipo	Dados
Carga por Professor	Barras	<code>carga_horaria_total</code> por professor (top 10)
Horas por Curso	Pizza	<code>carga_horaria_total</code> agrupada por curso
Uso de Laboratórios	Barras	<code>horas_laboratorio</code> por laboratório

Tabelas detalhadas

- **Relatório por Professor** — horas/semana, laboratório e sala principais.
- **Relatório por Laboratório** — horas de uso, horas ociosas e % de ocupação.
- Ambas exportáveis para **CSV** direto na interface.

Para validar a tela, o script `seed_bi.php` cria um quadro "Grade Demonstração BI" com aulas variadas (professores, cursos, labs e dias distintos), gerando distribuições visíveis em todos os gráficos.

12. Controle de Chaves

Gerencia a entrega e devolução das chaves dos laboratórios, registrando quem retirou, horário previsto e real de devolução e quem entregou/recebeu.

Fluxo:

- O **professor** registra a retirada (validada por posse da reserva e janela de horário).
- Enquanto não devolvida, a chave fica em_uso e aparece no painel do suporte.
- Se passar do horário previsto, o cartão é marcado como **ATRASADO** (com atalho de WhatsApp para cobrança).
- O **suporte** dá baixa informando quem recebeu e a hora real de devolução.

Detalhe técnico — ids sintéticos

A coluna `id_agendamento` de `controle_chaves` guarda tanto o id de uma reserva avulsa quanto um **id sintético** de uma aula da grade fixa (o id da aula + 1.000.000). Por isso a tabela não tem FK nessa coluna — ela referencia duas origens diferentes.

13. Chamados e Alerta Sonoro

Professores abrem chamados (gerais ou SOS) que entram em chamados_suporte com status pendente. O painel do suporte os exibe em tempo quase real.

Polling

A cada 8 segundos o painel chama `api/check-sos-status`, que devolve a contagem de pendentes e o HTML dos cartões de chamado (montados por `ApiController::checkSosStatus`), atualizando a fila sem recarregar a página.

Alerta sonoro

- Toca quando a contagem de chamados **aumenta** com o painel já aberto.
- O som fica em `public/assets/sounds/alerta-chamado.mp3` (local).
- Por política de autoplay dos navegadores, o áudio é "destravado" no primeiro clique/tecla do usuário; antes disso ele não pode tocar automaticamente.

14. Helpers

Helper	Responsabilidade
Auth	Sessão do usuário: check, id, nome, perfil, foto, login, logout, destinoAposLogin
Csrf	Token por sessão: token(), field() e check() (valida POST, HTTP 419 se inválido)
Upload	Upload de foto (extensões permitidas, remove a anterior, gera nome único)
HorarioHelper	Converte turno+período em horários; verifica choque de reservas
EnsalamentoHelper	Monta o mapa centralizado de ensalamento (salas, turnos, professores)

HorarioHelper — janelas de horário

Turno	Faixa padrão (sem período específico)
Matutino	08:20 – 11:55
Vespertino	14:20 – 18:00
Noturno	19:20 – 22:50

15. Camada de Dados (DAO / Model)

Cada agregado tem um contrato em DAOs/ e uma implementação PDO em DAOImpl/. Os models/ encapsulam operações de domínio que consomem os DAOs.

DAO / Model	Principais operações
UsuarioDAO	findByEmail, findById, create, listProfessores, updateFoto, upsertGoogle
AgendamentoDAO	solicitar/criar reserva, listar pendentes/confirmadas, histórico, status
ControleChaveDAO	registrarRetirada, darBaixa, emUso, historico, chavesPorProfessor
ChamadoSuporteDAO	abrir, resolver, pendentes, resolvidos, countPendentes
QuadroHorarioDAO	all, idAtivo, criar, duplicar, excluir, salvar/editar/mover/excluir aula
LaboratorioDAO	all, create, update, delete
DisciplinaDAO	all, create, update, excluir

16. Front-end

As views são templates PHP organizados por perfil (auth/, professor/, coordenador/, suporte/, shared/) sobre um layout unificado em `layouts/painel_open.php` e `painel_close.php`.

Recursos estáticos (public/assets)

- **css/app.css** — estilos do tema institucional (verde/laranja).
- **js/app.js** — token CSRF global, injeção do `_csrf` no submit, troca de seções do painel, tema.
- **images/** — logos UNICEPLAC e ícone do Google.
- **sounds/alerta-chamado.mp3** — alerta sonoro de chamados.

Bibliotecas via CDN

- Bootstrap 5.3 e Bootstrap Icons 1.11 (UI).
- Chart.js 4.4 (gráficos do BI).
- FullCalendar 6.1 (calendários de professor e coordenação).

17. Scripts de Banco de Dados

Script	Função
setup.php	Cria/recria TODO o schema (apaga tabelas existentes antes)
seed.php	Usuários demo + laboratórios, cursos, disciplinas, semestres, blocos/andares/salas
seed_bi.php	Recria o quadro "Grade Demonstração BI" com aulas variadas (idempotente)
migrate.php	Migrações incrementais idempotentes (ex.: coluna id_semestre)

Ordem recomendada de execução:

```
php backend/database/setup.php      # cria o schema
php backend/database/seed.php       # dados base + usuarios demo
php backend/database/seed_bi.php    # quadro de demonstracao do BI
php backend/database/migrate.php    # migracoes incrementais
```

18. Instalação e Execução

Pré-requisitos: PHP 8.2+ (com pdo_mysql), Composer, MySQL 8.x e, opcionalmente, Docker para o banco.

1) Dependências e ambiente

```
composer install
cp .env.example .env # edite as credenciais
```

2) Banco via Docker

```
docker run --name labhub-mysql \
-e MYSQL_ROOT_PASSWORD=labhub123 \
-e MYSQL_DATABASE=sistema_labs \
-p 3307:3306 -d mysql:8.4
```

No .env: DB_HOST=127.0.0.1, DB_PORT=3307, DB_USER=root, DB_PASS=labhub123.

3) Schema, dados e aplicação

```
php backend/database/setup.php
php backend/database/seed.php
php backend/database/seed_bi.php
php backend/database/migrate.php
php -S localhost:8000 -t public # acesse http://localhost:8000
```

Usuários de demonstração (senha 12345678):

Perfil	E-mail
Coordenador	coordenador@uniceplac.edu.br
Suporte	suporte@uniceplac.edu.br
Professor	professor@uniceplac.edu.br

19. Operação e Troubleshooting

Sintoma	Causa / Solução
Erro "Unknown column id_semestre"	Banco antigo sem a migração. Rode php backend/database/migrate.php.
"Sessão expirada ou requisição inválida" (HTTP 419)	Formulário POST sem token CSRF. Recarregue a página (o app.js injeta o _csrf no submit).
Alerta sonoro não toca	Política de autoplay: clique uma vez na página para destravar o áudio. Só toca em chamado novo.
Filtro de data joga para o login	Form GET precisa do campo page=suporte (corrigido como hidden input).
Front/back não abrem	Suba o MySQL (container) e a aplicação (php -S). Confira credenciais no .env.

20. Glossário

Termo	Significado
Ensalamento	Distribuição/alocação de turmas e disciplinas em salas por turno
Quadro de horários	Grade semanal de aulas fixas de um período letivo
Reserva avulsa	Agendamento pontual de laboratório feito pelo professor
Turno	Período do dia: Matutino, Vespertino ou Noturno
SOS	Chamado aberto pelo professor durante uma aula em andamento
BI	Business Intelligence — indicadores e gráficos analíticos
PRG	Post/Redirect/Get — padrão que evita reenvio de formulários
CSRF	Cross-Site Request Forgery — ataque mitigado por token por sessão
DAO	Data Access Object — camada de acesso a dados

— Fim da documentação —